```
  return (centroids,
clusterIndices)                                    #9
```

#1 Method `kmeans` takes a list of points and an integer, the number of clustering it should create (remember, one clustering per centroid...). We also pass `maxIter`, the maximum number of iterations. This function returns a pair, the list of centroids and the list of centroid indices associated to each point.
#2 Initialize the list of centroids. Different strategies can be used for random initialization, and also non-random initialization functions are viable (more on this in the next listings).
#3 Initialize list with the cluster index for each point. At first, each point belongs to the same big cluster containing the whole dataset.
#4 Repeat the main cycle (at most) `maxIter` times.
#5 Update the assignment of points to clusters. Stores the result in a temporary variable, to compare the new classification with the one from the previous iteration.
#6 If no point has switched cluster, the algorithm converged and so it can exit.
#7 Otherwise, copy over the assignments from the temporary variable.
#8 Update the centroids based on the new classification.
#9 Once the algorithm converged, we return both the centroids, and the assignments for each cluster.

To mitigate this issue, in practice k-means is always used with a random-restart strategy: the algorithm is run several times on a dataset, each time with a different random initialization of the centroids, and then results are compared